



南開大學
Nankai University

计算机学院
大数据计算及应用实验报告

任务一：推荐系统

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 6 月 20 日

目录

1 实验目的与要求	2
2 数据集说明与统计分析	2
2.1 文件组织与记录格式	2
2.2 基本规模、密度与稀疏度	3
2.3 评分分布	3
2.4 验证集与正式测试集规模	4
3 实验设计与评价方法	4
3.1 按用户留出验证集	4
3.2 RMSE 指标	5
3.3 时间、空间与可复现性	5
4 推荐算法原理	5
4.1 全局均值模型	5
4.2 用户/物品偏置基线	6
4.3 Item-Item 协同过滤	6
4.4 带偏置的潜因子矩阵分解	6
4.5 Item-CF 与矩阵分解融合	7
4.6 冷启动处理	7
5 程序实现	8
5.1 源码组织	8
5.2 完整实验流程	8
5.3 正式结果写出与校验	8
6 参数选择过程	9
6.1 矩阵分解参数对比	9
6.2 融合模型的融合权重选择	10
7 实验结果与分析	11
7.1 精度对比	11
7.2 训练时间、预测时间与空间消耗	12
7.3 时间与空间复杂度	13
8 最终模型与正式结果	14
8.1 最终模型确定依据	14
8.2 正式预测文件	14
8.3 结果正确性检查	15
9 复现实验与运行说明	15
10 总结与不足	16

课程实验报告：推荐系统评分预测

摘要

本实验围绕课程给定的用户-物品评分数据完成推荐系统评分预测。训练文件 `data/train.txt` 包含 **598 名用户、9077 个物品与 90854 条评分**，评分矩阵密度仅为 **1.6738%**，具有典型的高稀疏特征；测试文件 `data/test.txt` 包含 **610 名用户与 9982 个待预测的用户-物品对**。实验首先实现全局均值与用户/物品偏置基线，在统一的按用户留出的验证集上进一步实现 **Item-Item 协同过滤** 与带偏置、L2 正则化的**潜因子矩阵分解**，最后对两个模型进行**线性加权融合**。

实验以按用户留出 **20%** 评分构造验证集，固定随机种子为 **2026**，使用均方根误差 (RMSE) 评价预测精度，同时记录训练时间、验证预测时间与 Python 内存分配峰值。全局均值、偏置基线、Item-CF、矩阵分解与融合模型的验证 RMSE 依次为 **20.8541、17.4712、17.2198、17.4004 与 17.1500**。其中 Item-CF 是精度最高的单模型，但预测时间和相似度缓存开销较高；矩阵分解的预测速度快、空间占用较低；融合模型以 **0.6551 和 0.3449** 的权重分别加权两种预测，取得最低 RMSE。最终使用完整训练集重新训练融合模型，按要求生成了 `result.txt`，共输出 **9982 条连续评分预测**，并完成分组数量、用户/物品顺序、评分范围与文件哈希校验。

关键词： 推荐系统 Item-CF 矩阵分解 模型融合 RMSE 冷启动

1 实验目的与要求

本实验需要预测 `test.txt` 中所有用户-物品对的未知评分，并按 `ResultForm.txt` 的分组格式提交预测结果。作业要求中允许使用课程内外学习到的推荐算法，因此在本实验中，我们在完成单模型比较后，使用了**课程中介绍的 Hybrid Methods 思想构造融合模型**。

本实验的具体目标如下：

- 正确解析训练集和测试集的分组文本格式，统计用户数、物品数、评分数、与矩阵稀疏度等；
- 建立训练/验证划分及 RMSE 评价流程，避免直接使用无标签测试集调参；
- 实现全局均值、偏置基线、Item-CF、矩阵分解与融合模型，比较各模型的精度与资源开销；
- 对矩阵分解超参数和融合权重进行验证集选择，并说明选择依据；
- 处理测试集中出现的新用户、新物品与无有效邻居情况；
- 使用完整训练集生成最终结果文件，确保其用户顺序、物品顺序和记录数量与 `test.txt` 完全一致。

2 数据集说明与统计分析

2.1 文件组织与记录格式

本实验提供了 data 目录下的四个相关文件，其功能如下表所示。

文件	说明
train.txt	训练数据；用户头为 <user id> <count>，随后每行是 <item id> <score>。
test.txt	正式测试数据；用户头格式相同，随后每行只有待预测的物品编号。
ResultForm.txt	结果格式示例；需在每个测试物品编号后写入预测分数。
DataFormatExplanation.txt	训练、测试和结果记录格式说明。

表 1: 课程数据文件及作用

经过我们的测试程序逐行读取文件，并校验每个用户头中声明的记录数与实际记录数是否一致，发现训练集中不存在重复的用户-物品评分对，测试集也不存在重复待预测对；训练对与测试对无重叠。

2.2 基本规模、密度与稀疏度

设用户集合为 U 、物品集合为 I ，已知评分集合为 R 。评分矩阵理论上包含 $|U||I|$ 个位置，故矩阵密度和稀疏度定义为

$$\text{density} = \frac{|R|}{|U||I|}, \quad \text{sparsity} = 1 - \text{density}.$$

训练数据统计结果见下表表 2。

统计项	数值
用户数 $ U $	598
物品数 $ I $	9 077
已知评分数 $ R $	90 854
评分范围	10–100
平均评分	69.8821
矩阵密度	1.6738%
矩阵稀疏度	98.3262%

表 2: 训练数据基本统计

98.3262% 的矩阵位置没有评分，意味着仅依赖共同评分的邻域算法会面临有效邻居不足的问题，而参数较多的模型也容易在低频物品上过拟合。因此后续算法均保留基线预测作为 baseline，并在相似度或潜因子上使用正则化。

2.3 评分分布

训练评分取 10 的整数倍。图 2.1 展示各分值出现次数，其中 80 分最多，共 24 177 条；60、70、80 与 100 分占据主要部分，整体评分分布偏向较高分，这与平均分 69.8821 一致。该现象说明简单地固定预测 50 分并不合理，全局均值是更合适的最小基线。

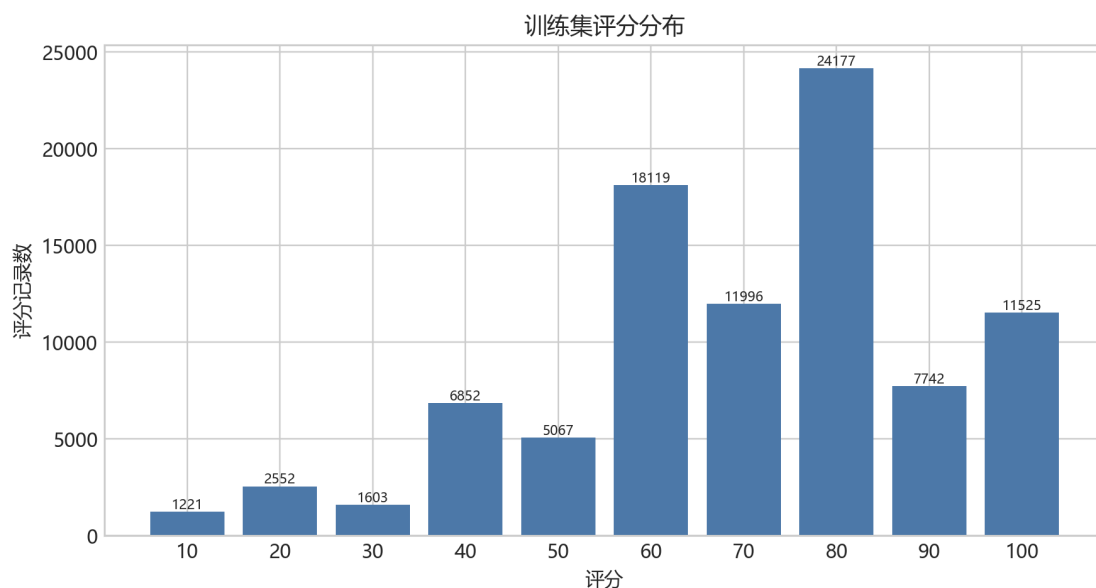


图 2.1: 训练集评分分布

2.4 验证集与正式测试集规模

按用户留出 20% 评分划分后，训练子集包含 72 687 条评分，验证集包含 18 167 条评分。正式测试集包含 610 个用户分组和 9 982 个待预测对。与完整训练数据比较，测试集中有 12 名新用户、647 个新物品；303 个测试对涉及新用户，659 个测试对涉及新物品。两类测试对可能重叠，不能直接相加。

统计项	数值
训练子集评分数	72 687
验证集评分数	18 167
验证集中涉及训练子集新物品的评分对	709 (3.90%)
正式测试用户数 / 预测对数	610 / 9 982
正式测试新用户数 / 新物品数	12 / 647
涉及新用户 / 新物品的测试对	303 / 659

表 3: 验证与测试数据及冷启动规模

3 实验设计与评价方法

3.1 按用户留出验证集

正式测试集 `test.txt` 没有真实评分，不能在本地直接计算测试 RMSE。为公平选择模型，**本实验对每名至少有两条评价的用户随机留出约 20% 评分作为验证数据**，同时保证该用户至少保留一条训练评分；少于两条评分的用户全部保留在训练子集中。随机种子固定为 2026，因此各模型面对完全相同的 72 687 条训练评分与 18 167 条验证评分，多次运行可复现。

这种按用户留出的方案比对全部评分无条件随机切分更贴近实际任务：系统已观察到用户的部分历史行为，需要预测其尚未观察的物品评分。划分只使用 `train.txt`，没有读取正式测试真实答案，避免测试信息泄漏。

3.2 RMSE 指标

设验证集中第 n 条真实评分为 r_n ，模型预测为 $\hat{r}_n$ ，验证记录数为 N ，则

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N (r_n - \hat{r}_n)^2}.$$

RMSE 对较大的预测误差惩罚更强，且与原评分处于相同量纲，数值越低表示评分预测越准确。对应实现如下。

RMSE 计算实现

```

1 def rmse(actual, predicted):
2     # 逐项计算平方误差并求均值
3     squared_error = sum(
4         (actual_score - predicted_score) ** 2
5         for actual_score, predicted_score in zip(actual, predicted)
6     )
7     # 开平方后恢复到原评分量纲
8     return sqrt(squared_error / len(actual))

```

该函数按相同顺序配对真实评分与预测评分，先计算均方误差，再开平方得到 RMSE。实验调用前会检查两组数据长度一致，因此分母可直接使用验证记录数。

3.3 时间、空间与可复现性

本实验我们的实验环境为 Windows 与 Python 3.8.0, 不依赖第三方推荐系统库。run_baselines.py 使用 time.perf_counter 测量训练和验证预测墙钟时间，使用 tracemalloc 记录 Python 内存分配峰值。由于内存跟踪本身会引入额外开销，表中的时间主要用于同一口径下的相对比较，不应理解为脱离硬件环境的固定性能；峰值内存也不等同于操作系统任务管理器中的进程总内存。

所有随机过程均使用种子 2026。最终结果文件连续两次生成的 SHA-256 均为

21c370718e7ffffefb363c5da1c815ff5c3ebbb42130e097e98d5c43d0ce41aaa2,

说明当前环境下输出具有逐字节可复现性。

4 推荐算法原理

4.1 全局均值模型

全局均值模型用训练集中所有评分的平均值 μ 预测任意用户-物品对：

$$\hat{r}_{ui} = \mu, \quad \mu = \frac{1}{|R|} \sum_{(u,i) \in R} r_{ui}.$$

其作用不是追求高精度，而是验证数据解析、划分与评价流程，并为复杂模型提供最低比较基准。本实验的完整训练集 train.txt 的均值为 69.8821。

4.2 用户/物品偏置基线

不同用户存在稳定的打分尺度差异，不同物品也存在整体受欢迎程度差异。偏置模型写为

$$\hat{r}_{ui} = \mu + b_u + b_i,$$

其中 b_u 为用户偏置， b_i 为物品偏置。程序采用交替更新，并在分母加入正则化系数 $\lambda_b = 10$ ：

$$b_u = \frac{\sum_{i \in R_u} (r_{ui} - \mu - b_i)}{\lambda_b + |R_u|}, \quad b_i = \frac{\sum_{u \in R_i} (r_{ui} - \mu - b_u)}{\lambda_b + |R_i|}.$$

共迭代 10 轮。分母中的正则项会将低频用户或物品偏置收缩到 0，减少少量评分导致的极端估计。

4.3 Item-Item 协同过滤

Item-CF 假设用户对相似物品具有相近的相对偏好。对物品 i, j ，先减去各自训练均值，再在共同评分用户集合 U_{ij} 上计算 Pearson 形式的中心化余弦相似度：

$$s_{ij}^{(0)} = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_j)^2}}.$$

共同评分人数很少时，相似度方差较大，因此采用收缩：

$$s_{ij} = s_{ij}^{(0)} \frac{|U_{ij}|}{|U_{ij}| + \lambda_s}, \quad \lambda_s = 10.$$

只有共同评分用户不少于 2 人的物品对才进入候选。对用户 u 的目标物品 i ，在该用户已评分物品中选择绝对相似度最大的 $K = 20$ 个邻居 $N(i; u)$ ，并在偏置基线上加权邻居残差：

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_{j \in N(i; u)} s_{ij} (r_{uj} - b_{uj})}{\sum_{j \in N(i; u)} |s_{ij}|}, \quad b_{ui} = \mu + b_u + b_i.$$

相似度按需计算，并使用容量为 250 000 的 LRU 缓存。该设计减少了预计算全部 9 077² 个物品对的空间，但将较多计算转移到预测阶段。

4.4 带偏置的潜因子矩阵分解

矩阵分解将用户和物品映射到 F 维潜在空间。用户 u 的向量为 $p_u \in \mathbb{R}^F$ ，物品 i 的向量为 $q_i \in \mathbb{R}^F$ ，预测公式为

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i.$$

训练仅在已知评分位置最小化正则化平方误差：

$$\min_{P, Q, b} \sum_{(u, i) \in R} [r_{ui} - (\mu + b_u + b_i + p_u^T q_i)]^2 + \lambda_f \sum_u \|p_u\|_2^2 + \lambda_f \sum_i \|q_i\|_2^2 + \lambda_b \sum_u b_u^2 + \lambda_b \sum_i b_i^2.$$

程序先用偏置基线初始化 μ, b_u, b_i ，随后用随机梯度下降联合更新偏置与潜因子。最终配置为 $F = 5$ 、训练 8 轮、初始学习率 0.002、每轮乘 0.95 衰减、潜因子正则化 0.5、偏置正则化 0.1。每个已知评分的计算量为 $O(F)$ ，训练后单次预测只需一个低维内积。

4.5 Item-CF 与矩阵分解融合

Item-CF 擅长捕捉局部物品关系，矩阵分解擅长提取全局低维结构，两者误差不完全重合。设其预测分别为 $p_{CF,n}$ 与 $p_{MF,n}$ ，采用

$$\hat{r}_n(\alpha) = \alpha p_{CF,n} + (1 - \alpha) p_{MF,n}, \quad 0 \leq \alpha \leq 1.$$

在验证集上最小化平方误差。由于 RMSE 是 SSE 的单调变换，二者得到相同最优权重。对 α 求导可得解析解

$$\alpha^* = \frac{\sum_n (r_n - p_{MF,n})(p_{CF,n} - p_{MF,n})}{\sum_n (p_{CF,n} - p_{MF,n})^2}.$$

计算得到 $\alpha^* = 0.6551032963$ ，故最终模型写为

$$\hat{r}_{ui} = 0.6551 \hat{r}_{ui}^{CF} + 0.3449 \hat{r}_{ui}^{MF}.$$

报告保留四位小数以便阅读，程序内部保留完整浮点精度。

4.6 冷启动处理

对于新用户、新物品及无有效邻居等冷启动场景，各模型采用以下兼容的兜底逻辑：

1. 用户和物品均已知时，使用对应完整模型；
2. Item-CF 找不到满足共同评分门槛的邻居时，退回 $\mu + b_u + b_i$ ；
3. 新物品没有物品偏置或潜因子时，使用 $\mu + b_u$ ；
4. 新用户没有用户偏置或历史邻居时，使用 $\mu + b_i$ ；
5. 用户和物品均未出现时，使用全局均值 μ 。

最终预测统一截断到训练评分范围 $[10, 100]$ ，防止残差加权或潜因子内积产生越界评分。

5 程序实现

5.1 源码组织

文件	功能
src/data_io.py	数据解析、按用户划分、统计及结果写出。
src/metrics.py	RMSE 指标。
src/baselines.py	全局均值与偏置基线。
src/item_cf.py	Item-CF 相似度、缓存和邻域预测。
src/matrix_factorization.py	带偏置和正则化的 SGD 矩阵分解。
src/ensemble.py	线性融合及解析最优权重。
run_baselines.py	统一运行单模型，记录 RMSE、时间与内存。
tune_matrix_factorization.py	矩阵分解候选参数比较。
tune_ensemble.py	融合权重选择及融合资源测量。
generate_predictions.py	完整训练、正式测试预测、结果校验和元数据输出。

表 4: 主要源码文件

5.2 完整实验流程

Algorithm 1 推荐模型训练、验证与正式预测流程

Input: 训练文件 R ，正式测试查询 T ，随机种子 2026

Output: 验证结果与最终预测文件

- 1: 解析 R ，统计用户、物品、评分分布与稀疏度
- 2: 按用户将 R 划分为训练子集 R_{tr} 和验证集 R_{val}
- 3: **for** 全局均值、偏置基线、Item-CF、矩阵分解 **do**
- 4: 在 R_{tr} 上训练，在 R_{val} 上预测并计算 RMSE
- 5: 记录训练时间、预测时间和峰值内存
- 6: **end for**
- 7: 在 R_{val} 上求 Item-CF 与矩阵分解的最优融合权重
- 8: 固定模型参数和融合权重，在完整 R 上重新训练两个模型
- 9: 预测 T ，按原用户/物品顺序写入 `result.txt`
- 10: 校验记录数、顺序、评分范围和 SHA-256

5.3 正式结果写出与校验

`write_predictions` 按 `test.txt` 的用户分组和物品顺序逐项调用最终预测函数，每个分数保留 6 位小数。写出后重新使用严格解析器读取结果，比较全部 (u, i) 序列，并检查有限性与 $[10, 100]$ 范围。关键融合代码如下。

最终融合预测核心代码

```

1 def predict(user_id, item_id):
2     # 分别获得两个单模型对同一用户--物品对的预测
3     item_prediction = item_cf.predict(user_id, item_id)
4     matrix_prediction = matrix_factorization.predict(user_id, item_id)

```

```

5  # 使用固定验证集确定的最优权重进行线性融合
6  return (
7      0.6551032963199183 * item_prediction
8      + 0.34489670368008174 * matrix_prediction
9  )

```

两个组件始终预测同一个用户-物品对，再按验证集确定的权重求加权平均。组件内部已执行冷启动处理和评分范围截断，因此该函数只负责组合预测结果，不再重新训练或调整权重。

6 参数选择过程

6.1 矩阵分解参数对比

矩阵分解首先尝试 20 个因子、25 轮和 0.005 学习率，但该配置的验证性能明显较差，验证 RMSE 为 20.1211，可能与学习率偏大、模型自由度较高及训练不稳定有关。因此实验降低学习率和因子数，增强正则化，并在固定验证集上比较 8 组候选。表 5 中 C5 最优。

编号	因子数	轮数	学习率	因子正则	偏置正则	RMSE
C1	10	8	0.0010	0.1	0.05	17.4464
C2	10	12	0.0010	0.5	0.10	17.4545
C3	20	8	0.0010	0.5	0.10	17.4639
C4	10	12	0.0005	0.1	0.05	17.4560
C5	5	8	0.0020	0.5	0.10	17.4004
C6	10	5	0.0020	0.5	0.10	17.4571
C7	10	8	0.0020	1.0	0.10	17.4418
C8	20	5	0.0015	1.0	0.10	17.4646

表 5: 矩阵分解候选参数与验证 RMSE

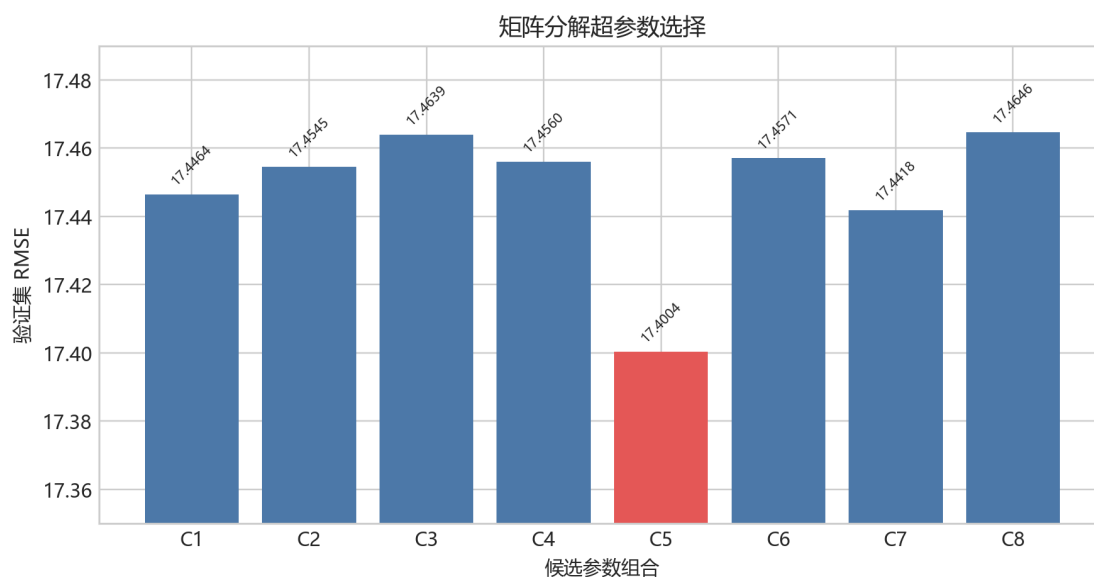


图 6.2: 矩阵分解超参数选择结果

结果表明，本数据物品多而评分稀疏，增加因子数没有改善验证误差，反而提高自由参数数量。5 个因子、8 轮和较强正则化在当前划分上取得较好泛化，因此作为最终矩阵分解配置。

6.2 融合模型的融合权重选择

图 6.3 先以 0.1 为步长扫描 Item-CF 权重。权重在 0.6–0.7 之间时 RMSE 最低：0.6 对应 17.1518，0.7 对应 17.1512。解析公式进一步得到 $\alpha = 0.6551032963$ ，对应 RMSE 17.1500。解析点落在网格最优区间内部，说明结果与粗粒度扫描一致，并非异常的浮点结果。

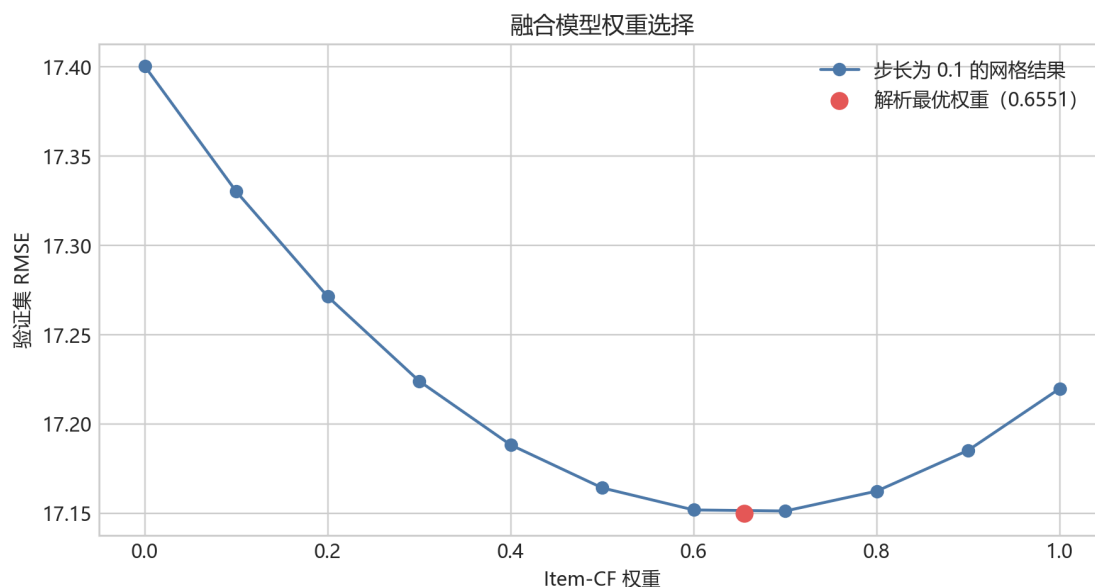


图 6.3: Item-CF 融合权重与验证 RMSE

```

output > summary > {} experiment_data_summary.json > {} final_prediction
116 "final_prediction": {}
117 "model": "item_item_cf_and_matrix_factorization_ensemble",
118 "weights": {
119   "item_item_cf": 0.6551032963199183,
120   "matrix_factorization": 0.34489670368008174
121 },
122 "configuration": {
123   "seed": 2026,
124   "item_item_cf": {
125     "neighbors": 20,
126     "minimum_common_users": 2,
127     "similarity_shrinkage": 10.0,
128     "similarity_cache_size": 250000
129   },
130   "matrix_factorization": {
131     "factors": 5,
132     "epochs": 8,
133     "learning_rate": 0.002,
134     "learning_rate_decay": 0.95,
135     "factor_regularization": 0.5,
136     "bias_regularization": 0.1,
137     "seed": 2026
138   }
139 },
140 "data": {
141   "training_ratings": 90854,
142   "test_users": 610,
143   "test_pairs": 9982,

```

图 6.4: 融合权重选择程序的实际运行结果 — 权重选择结果

7 实验结果与分析

7.1 精度对比

五种模型在相同验证集上的结果见表 6 与图 7.5。

模型	RMSE	相对结论
全局均值	20.8541	最小基线
偏置基线	17.4712	相对全局均值降低 16.22%
Item-CF	17.2198	相对偏置基线降低 1.44%
矩阵分解	17.4004	相对偏置基线降低 0.41%
融合模型	17.1500	相对偏置基线降低 1.84%

表 6: 各模型验证集精度

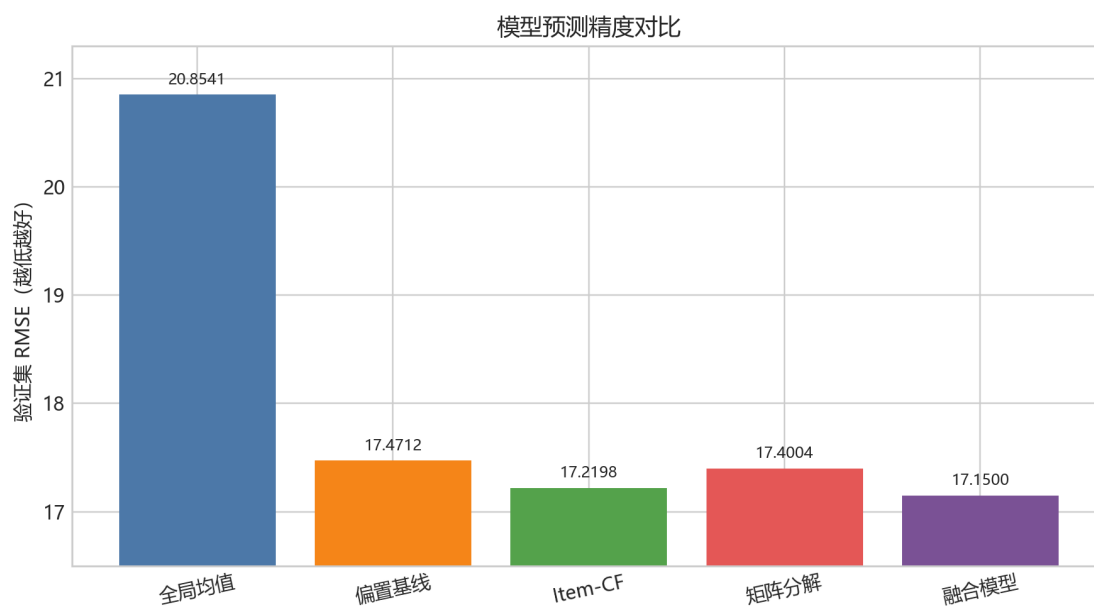


图 7.5: 各模型验证 RMSE 对比

偏置基线带来的提升最大，说明评分中的用户尺度差异与物品整体受欢迎程度是主要信号。Item-CF 进一步利用局部物品关系，成为最准确的单模型。矩阵分解虽未超过 Item-CF，但其误差结构与 Item-CF 不同，线性融合后 RMSE 降至 17.1500；相对全局均值累计降低 17.76%。

7.2 训练时间、预测时间与空间消耗

模型	训练时间/s	验证预测/s	峰值内存/MB
全局均值	0.0044	0.0076	0.15
偏置基线	1.3422	0.0626	10.39
Item-CF	1.5987	125.1781	81.10
矩阵分解	17.3838	0.1500	10.39
融合模型	16.7937	130.3994	81.16

表 7: 相同内存跟踪方法下的资源对比

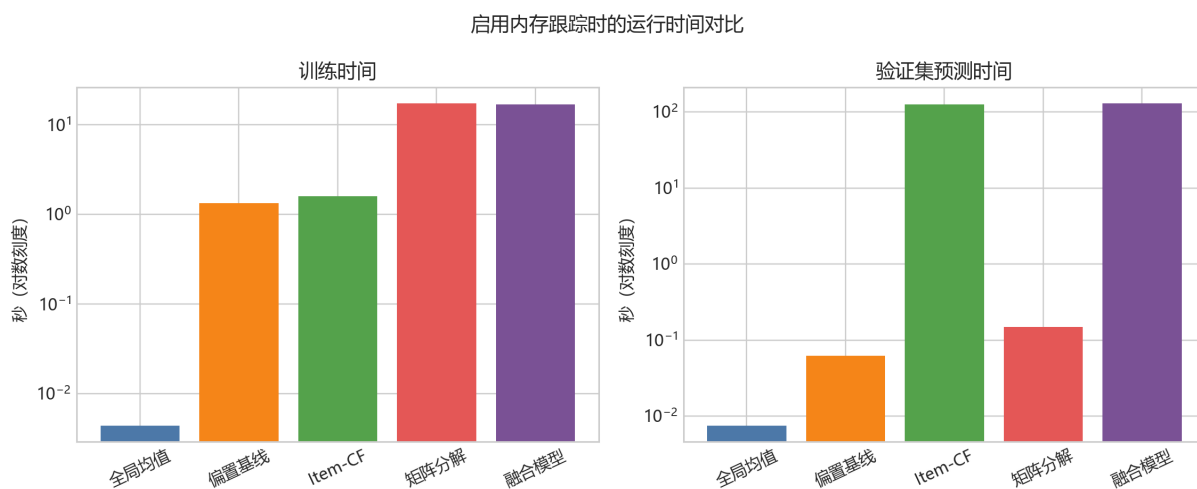


图 7.6: 训练与验证预测时间对比（纵轴为对数刻度）

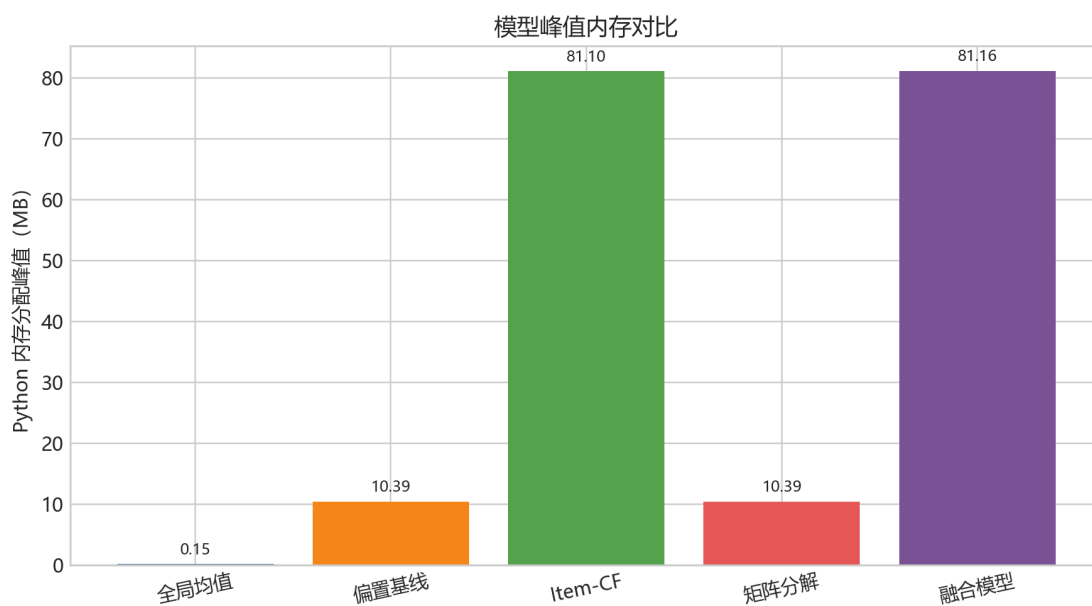


图 7.7: Python 内存分配峰值对比

Item-CF 的训练阶段只需建立索引与偏置, 因此较快; 但当前实现按需计算目标物品与用户历史物品的相似度, 验证预测需要 125.18 s, 并由相似度缓存将峰值内存推高到 81.10 MB。矩阵分解将主要成本放在训练阶段, 训练为 17.38 s, 但 18 167 条验证预测仅需 0.15 s。融合模型必须同时保留并运行两个组件, 资源开销由 Item-CF 主导。各模型采用相同的内存跟踪方法, 但数据来自不同次独立运行, 训练时间会受系统负载和测量波动影响, 因此融合模型的实测训练时间可能略低于单独运行矩阵分解时的结果。正式测试对数较少且不启用内存跟踪时, 完整结果写出约需 25.10 s。

```

output > experiments > {} baseline_results.json > {} results
51 "results": [
52   {
53     "model": "global_mean",
54     "rmse": 20.854125433454005,
55     "training_seconds": 0.004430900000000015,
56     "prediction_seconds": 0.007561199999999999,
57     "peak_memory_mb": 0.1527996063232422
58   },
59   {
60     "model": "bias_baseline",
61     "rmse": 17.471207300289958,
62     "training_seconds": 1.3422078,
63     "prediction_seconds": 0.06259339999999991,
64     "peak_memory_mb": 10.388427734375
65   },
66   {
67     "model": "item_item_cf",
68     "rmse": 17.21976241724554,
69     "training_seconds": 1.5986680000000002,
70     "prediction_seconds": 125.17808429999998,
71     "peak_memory_mb": 81.10053157806396
72   },
73   {
74     "model": "matrix_factorization",
75     "rmse": 17.400366069219004,
76     "training_seconds": 17.3838389,
77     "prediction_seconds": 0.14999760000000006,
78     "peak_memory_mb": 10.385719299316406
  ]

```

图 7.8: 全部单模型对比实验的实际运行结果

7.3 时间与空间复杂度

设评分数为 $|R|$, 用户数、物品数分别为 $|U|, |I|$, 潜因子数为 F , 训练轮数为 E , Item-CF 邻居数为 K , 相似度缓存容量为 C 。

模型	训练复杂度	单次预测主要复杂度	主要空间复杂度
全局均值	$O(R)$	$O(1)$	$O(1)$
偏置基线	$O(E_b R)$	$O(1)$	$O(U + I)$
Item-CF	$O(R + E_b R)$	与 $ R_u $ 个候选及共同用户求交有关, 再选 Top- K	$O(R + C)$
矩阵分解	$O(E R F)$	$O(F)$	$O((U + I)F)$
融合模型	两个组件训练之和	两个组件预测之和	两个组件空间之和

表 8: 模型复杂度分析

从理论和实测两方面均可看出: 矩阵分解适合大量在线预测; 当前 Item-CF 更适合作为离线实验模型。若面向生产系统, 可离线预计算每个物品的 Top- K 相似物品、使用稀疏矩阵向量化或近似近邻索引, 以换取更快预测。

8 最终模型与正式结果

8.1 最终模型确定依据

最终选择 Item-CF 与矩阵分解融合，而不是单独选择 RMSE 最低的 Item-CF，原因包括：

- 融合模型验证 RMSE 为 17.1500，低于所有单模型；
- 权重由固定验证集上的解析最小二乘得到，并由 0.1 步长网格扫描进一步验证其合理区间；
- Item-CF 与矩阵分解分别建模局部邻域和全局潜因子，具有互补性；
- 实践了我们在课上介绍的 Hybrid Methods 与 Netflix Prize 中的模型组合的思想。

模型选择和权重确定后，不再使用正式测试数据调参；完整 `train.txt` 用于重新估计模型参数。

8.2 正式预测文件

执行命令如下：

生成最终结果文件

```
1 # 在完整训练集上训练融合模型并生成 result.txt
2 python generate_predictions.py
```

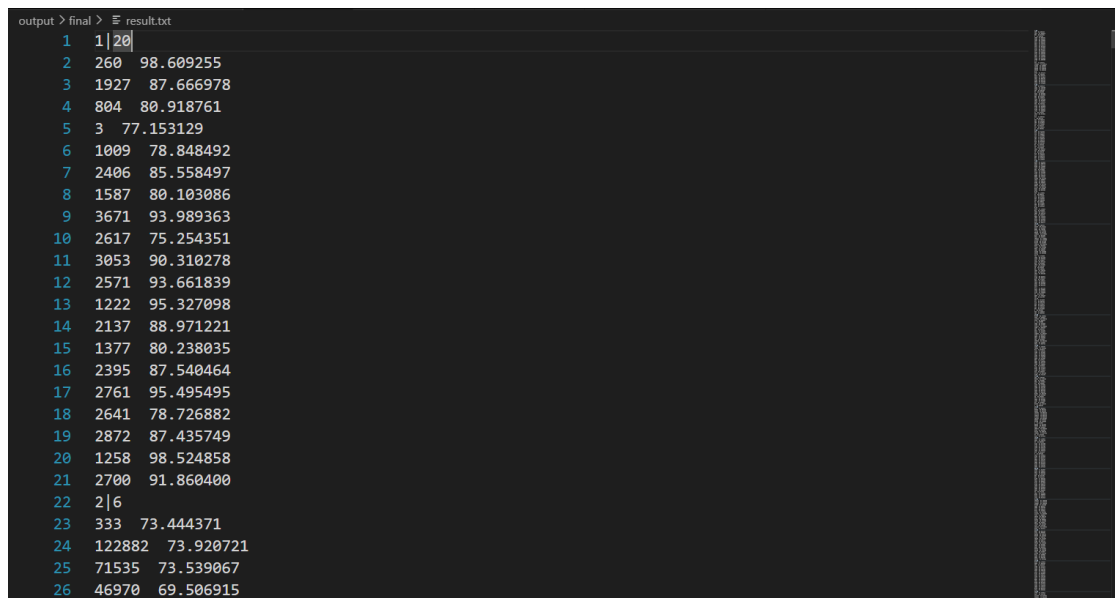
程序在完整 90854 条评分上训练两个组件，预测 9982 个测试对并写入 `output/final/result.txt`。文件共有 610 个用户头和 9982 个评分行，总计 10592 行；预测最小值为 24.870695，最大值为 100.000000，平均值为 71.459618。预测分数保留 6 位小数。由于正式测试集不提供真实评分标签，因此无法在本地计算最终测试 RMSE；报告中的 17.1500 均指固定验证集上的 RMSE，而不是正式测试集成绩。

```
output > final > {} result_metadata.json > {} data
7   "configuration": {
15     "matrix_factorization": {
20       "factor_regularization": 0.5,
21       "bias_regularization": 0.1,
22       "seed": 2026
23     }
24   },
25   "data": {
26     "training_ratings": 90854,
27     "test_users": 610,
28     "test_pairs": 9982,
29     "new_test_users": 12,
30     "new_test_items": 647
31   },
32   "timing_seconds": {
33     "item_item_cf_training": 0.6940763999999999,
34     "matrix_factorization_training": 5.846769500000001,
35     "prediction_and_writing": 25.0994099
36   },
37   "validation": {
38     "predictions": 9982,
39     "minimum_prediction": 24.870695,
40     "maximum_prediction": 100.0,
41     "mean_prediction": 71.45961843277838,
42     "sha256": "21c370718e7fffebf363c5da1c815ff5c3ebb42130e097e98d5c43d0ce41aaa2"
43   }
44 }
```

图 8.9: 正式预测文件的生成与校验结果

模型权重、训练数据量、测试对数量、运行时间和 `validation` 校验信息，证明正式结果文件由既定模型完整生成。

8.3 结果正确性检查



output > final > result.txt
1 1 20
2 260 98.609255
3 1927 87.666978
4 804 80.918761
5 3 77.153129
6 1009 78.848492
7 2406 85.558497
8 1587 80.103086
9 3671 93.989363
10 2617 75.254351
11 3053 90.310278
12 2571 93.661839
13 1222 95.327098
14 2137 88.971221
15 1377 80.238035
16 2395 87.540464
17 2761 95.495495
18 2641 78.726882
19 2872 87.435749
20 1258 98.524858
21 2700 91.860400
22 2 6
23 333 73.444371
24 122882 73.920721
25 71535 73.539067
26 46970 69.506915

图 8.10: result.txt 结果

结果生成后自动完成以下校验：

1. 用户分组数为 610，预测记录数为 9982；
2. 所有用户和物品的顺序与 `test.txt` 完全一致；
3. 所有预测均为有限数，且位于 $[10, 100]$ ；
4. 结果文件 SHA-256 固定，可检测意外修改；
5. 6 项自动化测试全部通过，覆盖数据解析、可复现划分、RMSE、冷启动、Item-CF、矩阵分解和融合公式。

9 复现实验与运行说明

主要命令如下：

实验复现命令

```
1 # 运行全部单模型对比实验
2 python run_baselines.py
3
4 # 搜索矩阵分解的候选参数
5 python tune_matrix_factorization.py
6
```



```
7 # 选择 Item-CF 与矩阵分解的融合权重
8 python tune_ensemble.py
9
10 # 使用完整训练集生成最终结果文件
11 python generate_predictions.py
12
13 # 运行全部自动化测试
14 python -m unittest discover -s tests -v
```

上述命令依次覆盖单模型比较、参数选择、融合权重确定、正式结果生成和自动化测试。调参程序只使用固定验证集，最终结果生成阶段才重新使用完整训练集，避免正式测试数据参与模型选择。

`run_baselines.py` 默认运行全部模型，因 Item-CF 动态计算相似度，在启用内存跟踪时约需两分钟。若只需快速复现矩阵分解，可执行

```
1 # 仅运行矩阵分解，适合快速检查环境和数据读取
2 python run_baselines.py --models matrix_factorization
```

该命令跳过耗时较长的 Item-CF 验证预测，仅用于快速复现矩阵分解结果，不能代替完整的多模型对比实验。

实验 JSON 分别保存在 `output/experiments/`，最终预测与元数据位于 `output/final/`。

10 总结与不足

本实验从可复现的最小基线**全局均值模型**出发，逐步完成**用户/物品偏置基线**、**Item-Item 协同过滤**、带正则化的潜因子**矩阵分解与线性模型融合**。实验验证了以下结论：用户和物品偏置是最主要的可解释信号；**Item-CF** 在当前数据上取得**最佳单模型精度**，但动态相似度计算带来显著预测开销；矩阵分解具有快速预测和紧凑存储的优势；**两者融合**利用误差互补性，**将验证 RMSE 进一步降低到 17.1500**。完整流程还覆盖了冷启动兜底、参数选择、资源测量、正式结果校验和可复现性验证，满足实验对基本统计、算法细节、RMSE、训练时间、空间消耗和实验分析的要求。

但是我们的当前实验仍存在以下不足：仅采用一次固定按用户留出划分，模型选择可能受该划分影响；Item-CF 尚未离线预计算 Top-K 邻居；测试中的 647 个新物品缺少内容特征，只能依赖偏置兜底；融合仅使用一个全局线性权重。后续可以增加多次交叉验证、使用物品内容或类别缓解新物品冷启动、预计算相似邻居，并研究按用户活跃度或物品流行度自适应的融合权重。

注：github 仓库

代码详见：<https://github.com/Wangxuueuuu/Big-Data-Computing-and-Applications>